Unidad 105: Shell Scripts y manejo de datos

- 105.1: Customizar y usar el entorno de Shell (4)
- 105.2: Customizar y escribir scripts (4)
- 105.3: Gestión de datos SQL (2)

105.1: Customizar y utilizar el Entorno Shell (Peso: 4)

Variables de entorno

Las variables de entorno proporcionan los medios para pasar los datos mencionados (variables) a los programas lanzados desde una shell, propios depósitos también se basan en el medio ambiente

Establecer una variable de entorno como se acaba de describir para todos los programas que inicia de la misma se utiliza ejecutando **env**

105.1: Customizar y utilizar el Entorno Shell (Peso: 4)

Variables de entorno

Se pueden encontrar muchas variables de entorno comunes en el sistema. Para ello, el comando **env** devuelve todas las variables de entorno que ya están definidas

Si queremos quitar el valor de una variable de entorno, nos basta con usar **unset nombre_variable**

105.1: Customizar y utilizar el Entorno Shell (Peso: 4)

Alias

En Linux es posible la ejecución de diferentes Alias para poder simplificar algunos comandos. Para implementar un alias, podemos ejecutar lo siguiente:

alias ll = 'ls -la'

105.1: Customizar y utilizar el Entorno Shell (Peso: 4)

Archivos de configuración de shell

Configurar shell requiere editar archivos de configuración de perfiles de Shell. Los archivos de configuración lo podemos dividir en Globales o de Usuarios y en Login files (lanzados por el usuario al loguearse) o no Login Files (lanzados por algún proceso)

Global: /etc/profile, /etc/profile.d /etc/bashrc
User: /.bash_login, /.bash_profile /.bashrc .bash_logout

105.2: Customizar y escribir Scripts (Peso: 4)

Escribiendo Scripts

Si necesitas añadir 1000 usuarios no es viable ni práctico ejecutar 1000 veces el comando useradd, para ello, realizamos scripts en línea de comandos para ayudarnos a la automatización de los programas.

Los Shell Scripts son simples programas escritos en un lenguaje de programación interpretado que se incrustan en el shell de Linux que se utiliza para escribir comandos.
La mayoría de sistemas Linux usan bash por defecto, por lo que los scripts de shell se escriben a menudo en bash

105.2: Customizar y escribir Scripts (Peso: 4)

Empezando Shell Scripting

Si vemos cualquier script o fichero de init vemos que la mayoría empiezan con la lína **#!/bin/bash**

Los dos primeros caracteres son un código especial que indica al kernel Linux que se trata de una secuencia de comandos y utilizar el resto de la línea como una ruta al programa que es interpretar el guión. (shebang). El lenguaje de programación Shell utiliza una almohadilla (#) como carácter de comentario, por lo que la utilidad de la escritura ignora esta línea, aunque el kernel no.

105.2: Customizar y escribir Scripts (Peso: 4)

Empezando Shell Scripting

Para que un shell script se pueda ejecutar debe tener permisos de ejecución. Como vimos en el capítulo anterior: **chmod a+x**

Para ejecutar el script, lo podemos ejecutar con el **./shell_script** o **sh shell_script**. También podemos usar el comando **source shell_script**

105.2: Customizar y escribir Scripts (Peso: 4)

Usando comandos y variables

En cada Shell es de mucha utilidad usar comandos del propio Linux para manipular logs, archivos, carpetas, usuarios,etc. Los comandos más usados son:

- grep
- find
- cut
- sed
- echo
- mail

105.2: Customizar y escribir Scripts (Peso: 4)

Usando comandos y variables

Una variable el valor que le damos a un determinado nombre para poder trabajar con ella durante todo un proceso de ejecución. Así mismo, en Shell, podemos identificar variables

Ejemplo:

```
DIAS_A_MANTENER=6
PATH=/usr/bin/mysqldump
```

105.2: Customizar y escribir Scripts (Peso: 4)

Usando comandos y variables

Las variables también se pueden leer durante tiempo de ejecución:

```
#!/bin/bash

echo “Di un número:”
read a
echo “El número es:”$a
```

105.2: Customizar y escribir Scripts (Peso: 4)

Arrays

A una variable le podemos asignar tambien un conjunto de valores (Array)

Ejemplo:

```
ciudad[0]="Madrid"
ciudad[1]="Tokyo"
ciudad[2]="Berlin"
```

Luego puedes llamar uno de los valores o todos:

```
echo ${ciudad[2]} → Berlin
echo ${ciudad[*]} → Madrid Tokyo Berlin
```

105.2: Customizar y escribir Scripts (Peso: 4)

Variables especiales

$? : Valor devuelto por el último programa que se ha ejecutado.

$! : Número de proceso (PID) del último proceso que se ejecuto en background.

$$: Número de proceso (PID) de la shell actual.

$PATH : Contiene una lista separada por dos puntos de directorios en los cuales el intérprete de comandos buscará los archivos ejecutables que no se invocan con una ruta.

$HOME : Contiene la ubicación del directorio de usuario.

105.2: Customizar y escribir Scripts (Peso: 4)

Usando parámetros de entrada

Un script en Shell no deja de ser un programa, luego también tiene sus propios programas de entrada

Ejemplo:

sh elimina_archivos log 5

En este ejemplo, nuestro script tiene los siguientes parámetros:

- elimina_archivos=$0
- log=$1
- 5=$2

105.2: Customizar y escribir Scripts (Peso: 4)

Usando parámetros de entrada

En nuestro script, las variables serán $0, $1 y $2

Podemos hacer para comprobar su valor,

echo $0
echo $1
echo $2
Echo $#
$@ → Todos los argumentos
$# → Número de argumentos que se han pasado al programa.

Con **set** podemos asignarles nuevos valores

105.2: Customizar y escribir Scripts (Peso: 4)

Condiciones

Los lenguajes de scripting también pueden trabajar con condiciones.
Las condiciones que se suelen usar son **if else** . Hay diferentes condiciones (**test**):

-f: si el archivo existe
-s: si el archivo existe y es mayor de cero
&&: ambos lados deben cumplirse
||: uno de los lados deben cumplirse
=: comparando cadenas

105.2: Customizar y escribir Scripts (Peso: 4)

Comparacion de cadenas alfanumericas

OPERADOR	Verdad (valor TRUE) si
cadena1=cadena2	cadena1 es igual a cadena2
cadena1!=cadena2	cadena1 es diferente a cadena2
cadena1<cadena2	cadena1 es menos que cadena2
cadena1>cadena2	cadena1 es mayor que cadena2
-n cadena1	cadena1 no es nula (longitud mayor que 0)
-z cadena1	cadena1 tiene un valor nulo (longitud 0)

105.2: Customizar y escribir Scripts (Peso: 4)

Comparacion de valore numericos

OPERADOR	Verdad (valor TRUE) si
varx -lt vary	varx menor que vary
varx -le vary	varx menor o igual que vary
varx -eq vary	varx igual que vary
varx -ge vary	varx mayor o igual que vary
varx -gt vary	varx mayor que vary
varx -ne vary	varx diferente que vary

105.2: Customizar y escribir Scripts (Peso: 4)

Condiciones

Ejemplo:

```
if [ -s /tmp/tempstuff ]
then
    echo “/tmp/tempstuff found; aborting!”
exit
fi
```

También podemos comparar con comandos:

```
If [ command ] then //action// fi
```

105.2: Customizar y escribir Scripts (Peso: 4)

Condiciones

También con expresiones condicionales.

```
if [ expresión-condicional ]
then
      //acciones o comandos
Elif [ condición ]
Then
//acciones
else
         // otros comandos
fi
```

105.2: Customizar y escribir Scripts (Peso: 4)

Condiciones

También con expresiones condicionales.

```
Case $VAR in
OPCION1)
;;
OPCION2)
;;
*)
;;
esac
```

105.2: Customizar y escribir Scripts (Peso: 4)

Bucles

Los bucles nos permiten iteraciones para poder repetir una acción. En Shell, podemos hacer bucles con ficheros

```
#!/bin/bash

for d in `ls *.log` ; do
    gzip -9f $d
    done
```

105.2: Customizar y escribir Scripts (Peso: 4)

Bucles

Para uso de bucles, podemos usar también la opción **while** que se ejecuta mientras se cumpla una condicion

```
while [ condicion ]
do
    comandos
Done
```

Ejemplo desde la shell

while true; do free -m; sleep 100; done

105.2: Customizar y escribir Scripts (Peso: 4)

Bucles

Para uso de bucles, podemos usar también la opción **until,** que se ejecuta hasta que se cumpla una condicion

```
until [ condicion ]
do
    comandos
Done
```

Ejemplo desde la shell

num=1; until [num -eq 5]; do let num=num+1; echo $num; done

105.2: Customizar y escribir Scripts (Peso: 4)

Bucles

También podemos usar secuencias de números a imprimir con el comando **seq** . Nos imprimirá números según la secuencia que le demos.

Ejemplo:

seq 1 5

for i in `seq 1 10`; do `ab -n 7 -c 5 http://ip_test/` done

105.2: Customizar y escribir Scripts (Peso: 4)

Menús → SELECT

SELECT nos permite seleccionar opciones entre opciones a modo de menú

```
echo "¿que quieres?"

select accion in comer dormir beber leer
do
case $accion in
        comer)  echo "ve a la nevera" ;;
        dormir) echo "vete a la cama" ;;
        beber)  echo "vete al bar" ;;
        leer)   echo "vete a la biblioteca" ;;
esac
done
```

105.2: Customizar y escribir Scripts (Peso: 4)

GETOPTS

Nos permite especificar modificadores para nuestro script

```
#!/bin/bash
while getopts "abc" opts
do
case $opts in
        a) echo "has usado a" ;;
        b) echo "has usado b" ;;
        c) echo "has usado c" ;;
esac
done
```

105.2: Customizar y escribir Scripts (Peso: 4)

Operaciones Aritméticas

let → utiliza variables como numeros

expr → realiza calculos directamente

bc → permite el uso de decimales con “scale=x” o con -l

105.2: Customizar y escribir Scripts (Peso: 4)

Funciones

Las funciones son muy útiles cuando queremos modularidad y organización. Lo mejor es verlo con un ejemplo:

```
#/bin/bash
doit() {
      cp $1 $2
}
function check() {
 if [ -s $2 ]
 then
       echo “Target file exists! Exiting!”
       exit
  fi
}
check $1 $2
doit $1 $2
```

105.2: Customizar y escribir Scripts (Peso: 4)

EJERCICIOS

1. Borrar todos los archivos de un directorio de forma listada pero que nos pregunte si lo queremos borrar o no
2. Leer de un fichero de entrada un listado de usuarios y crear un directorio por cada uno de ellos. Dar permisos de lectura y escritura SÓLO al propietario.
3. Script que nos diga la memoria del sistema actual y la lleva a un log cada 5 min.
4. Hacer un script que corra en background que vaya comprobando si se ha mapeado un disco o no
5. Crear script que nos diga si hay más de un usuario conectado a mi sistema.

105.3: Gestión de datos SQL (Peso: 2)

SQL (Structured Query Languaje)

Lenguaje declarativo de acceso a las Bases de Datos relacionales del cual nos permite especificar diversos tipos de operaciones en ellas.

Los datos que nos devuelven se organizan mediante tablas para que podamos trabajar mejor con ellas.

105.3: Gestión de datos SQL (Peso: 2)

SHOW DATABASES

Con el siguiente comando vemos las Bases de Datos que están actualmente creadas en el sistemas. La salida debe ser del tipo:

```
mysql> SHOW DATABASES;
+--------------------+
| Database |
+--------------------+
| information_schema |
+--------------------+
1 row in set (0.00 sec)
```

105.3: Gestión de datos SQL (Peso: 2)

CREATE DATABASE

Con el comando CREATE DATABASE database_name; creamos una base de datos con el nombre que hayamos definido

USE database

Una vez creada, para poder usarla y crear registros sobre ella escribimos USE database_name

105.3: Gestión de datos SQL (Peso: 2)

CREATE TABLE

Para crear una tabla sobre la Base de Datos

CREATE TABLE ejemplo (nombre VARCHAR(30), dni VARCHAR (30), edad INTEGER, ciudad VARCHAR(20));

105.3: Gestión de datos SQL (Peso: 2)

INSERT TABLE

Para insertar valores en una tabla de una Base de Datos

INSERT INTO ejemplo VALUES (“FHF”,”80012312L”,35,”BARCELONA”);

También podemos actualizar los datos de una tabla

UPDATE TABLE

UPDATE ejemplo SET edad=25 WHERE DNI=“80012312L”

105.3: Gestión de datos SQL (Peso: 2)

SELECT

Para sacar la información que hay en nuestras Tablas de las Bases de Datos usamos el comando SELECT

Sacamos todoslos registros de la tabla ejemplo:
SELECT *from ejemplo;

Sacamos todos los registros pero sólo el valor DNI de la tabla ejemplo:
SELECT DNI from ejemplo;

105.3: Gestión de datos SQL (Peso: 2)

SELECT

Con Select podemos hacer consultas a múltiples tablas. Vamos primero a crear otra tabla y a darle registros:

```
CREATE table ejemplo2 (id INTEGER, nombre VARCHAR(30),edad INTEGER);
INSERT INTO ejemplo2 VALUES(1,”EJEMPLO2”,18);

SELECT ejemplo1.DNI FROM ejemplo1 AS e1, ejemplo2 AS e2  WHERE e1.edad=e2.edad;
```

105.3: Gestión de datos SQL (Peso: 2)

WHERE

La cláusula WHERE nos permite filtrar según una condición ya esas númerica o de texto o el resultado de otra tabla.

105.3: Gestión de datos SQL (Peso: 2)

DELETE

Para borrar datos usamos la sentencia DELETE

DELETE FROM ejemplo2 WHERE edad>45;

Nos borrará todos los registros mayores de 45

105.3: Gestión de datos SQL (Peso: 2)

EJERICICIOS

Instalar de repositorio el servicio MySQL y arrancarlo
Conectarnos usando el mysql client
Crear un par de Bases de Datos
Crear Registros para esas Bases de Datos
Realizar operaciones de consulta y borrado

105.3: Gestión de datos SQL (Peso: 2)

GROUP BY

Group by se usa para agrupar una salida en función del número de registros

SELECT nombre, edad FROM ejemplo1 GROUP BY edad;

105.3: Gestión de datos SQL (Peso: 2)

ORDER BY

En MySQL además tenemos la opción de ordenar las tablas en función de un campo.

SELECT * from ejemplo WHERE edad>15 ORDER BY edad;

105.3: Gestión de datos SQL (Peso: 2)

JOIN

Hay diferentes tipos de JOIN en MySQL (LEFT, RIGH e INNER)

SELECT * from ejemplo 1 a, ejemplo2 b where a.edad>b.edad;

Esto es lo mismo que:

SELECT `* from ejemplo 1 a INNER JOIN ejemplo2 b on a.edad=b.edad;

A esto lo llamamos JOIN NATURAL

105.3: Gestión de datos SQL (Peso: 2)

JOIN

LEFT JOIN es una combinación externa de las dos tablas donde saca el conjunto más todos los datos de la tabla de la izquierda

SELECT * from ejemplo 1 a left join emeplo2 b on a.edad=b.edad;

RIGHT JOIN funciona igual que LEFT join pero sacando el conjunto de la derecha por completo